```
  sandwich.toppings << 'cheese'
  toppings = sandwich.toppings

  expect(toppings).not_to be_empty
end
```

The expectation for this example introduces two new twists. First, you can negate your expectation—that is, check for falsehood—by using `not_to` instead of `to`. Second, you can test that a collection such as an `Array` or `Hash` is empty using `be_empty`. You'll see more about how these constructs work in *Exploring RSpec Expectations*.

Now, run your new example:

```
$ rspec
..

Finished in 0.00201 seconds (files took 0.09252 seconds to load)
2 examples, 0 failures
```

This spec runs fine, but it's a little repetitive. We're copying the sandwich setup code in each example. This duplication makes it harder to change the common code later on. It also clouds up our examples with setup information.

Let's make a common sandwich available to all our tests. RSpec supports multiple ways to do so:

- RSpec *hooks* run automatically at specific times during testing.
- *Helper methods* are regular Ruby methods; you control when these run.
- RSpec's *let* construct initializes data on demand.

Each of these techniques has its advantages; you'll use them all as you follow along with the examples in the book. Let's take a look at each one in turn to see how you'd use them.

Hooks

The first thing we'll try is an RSpec *before hook*, which will run automatically before each example. Add the following highlighted line inside your example group, just inside the `RSpec.describe` block:

01-getting-started/05/spec/sandwich_spec.rb

```
RSpec.describe 'An ideal sandwich' do
  before { @sandwich = Sandwich.new('delicious', []) }
```

RSpec keeps track of all the hooks you've registered. Each time RSpec is about to start running one of your examples, it will run any `before` hooks that apply. The `@sandwich` instance variable will be set up and ready to use.